The `fca_math` enlightener

A thoughtful introduction to the mathematics of Formal Concept Analysis, its style and guiding principles, and a `Python` software package.

Bernhard Ganter
Ernst-Schröder–Zentrum Darmstadt

December 10, 2025

Formal Concept Analysis is a mathematical theory, but it tastes a little different. Mathematicians sometimes have difficulty with this. Not because it does not follow the rules. Formal Concept Analysis has definitions, theorems, proofs, applications and open problems, just like any other mathematics. But FCA imposes additional rules on itself. These relate to the technical terms, examples and visualisations used. Some consider this superfluous, but we believe it is essential for the (modest) success of the theory. Admittedly, it takes some getting used to. That is why this text is not written like a mathematical textbook. It provides additional information and recommendations throughout. You do not have to follow them, but it is worth it.

Couldn't I have just written a standard mathematical text? In fact, I did just that some time ago and, together with Rudolf Wille, wrote a monograph on the mathematical foundations of Formal Concept Analysis. The second edition was recently published. That book is considered a standard work, is often referred to and occasionally commended. But I have my doubts as to whether it is read as often as it is cited. My impression is that many authors derive their knowledge from much simpler short introductions to the field, which are usually written from a specific perspective, which, hwever, also narrows the view.

And one more thing: this text also introduces to my package `fca_math.py`. Some time ago, I started porting my computer program collection from `Pascal` to `Python`. My implementation is not particularly fast. I have already been told that it is not suitable for professional data analysis. But that is not what it is for. There are already a considerable number of Python programs for Formal Concept Analysis, and some of them may be more effcient. My interest lies in the mathematical foundations, and this collection of programs allows me (and you) to explore many of the theoretical results without much effort.

1 First questions and answers



Q: Why questions and answers?

A: Did you carefully read the introduction on the previous page? No? That's right: the text was too long and therefore boring. We do not expect our readers to work through this manuscript linearly from start to finish. Sections having a blue heading should not be skipped.

Q: Is Formal Concept Analysis just another math speciality?

A: I believe that Formal Concept Analysis is particularly *meaningful*.

Formal Concept Analysis is based on a mathematization of *concept* and *conceptual hierarchy*. Concepts are fundamental to cognition, comparable, e.g., to *spatial* thinking, which is mathematized by geometry. This touches on philosophical, psychological, and linguistic questions that go beyond the scope of this text, but are crucially important for understanding Formal Concept Analysis. We will therefore return to such aspects repeatedly.

Q: Why does Formal Concept Analysis have such a clumsy name?

A: It was clear from the outset that the significance of Formal Concept Analysis is not primarily internal to mathematics. Therefore, the technical language must be more generally understandable. To claim that concepts can be defined and explained mathematically would rightly be rejected. The mathematical objects that result from the mathematization of *concept* are not concepts themselves and should therefore be called something else, *formal concepts*.

2 What the reader should already know

This text introduces a mathematical field of work. Therefore, we assume that readers are familiar with basic mathematical texts. Simple terms from set theory should be understood. No further specialized knowledge is required, except perhaps in examples, which may be skipped.

In fact, experienced mathematicians sometimes find it more difficult than others to understand Formal Concept Analysis, because our technical language is somewhat different, more meaning-oriented.

The concept lattices that will be of central importance in the following are (partially) ordered sets with special properties. However, this will be explained in detail. The diagrams commonly used for visualization will also be introduced.

If you want to use the associated Python package `fca_math.py` as recommended, you should have access to a Python3 installation and know how to run it. Most of our programs are very simple, short, and open to improvement.

3 Getting started

Figure 1 is based on original medical data from a treatment of an eating disorder, Anorexia nervosa. The therapists were interested in the patient's view of her social environment because they considered it significant. The patient therefore distinguished between several people based on attributes she chose.

	over-sensitive	*withdrawn*	*self-confident*	*dutiful*	*cordial*	*difficult*	*attentive*	*easily offended*	*calm*	*apprehensive*	*chatty*	*superficial*	*sensitive*	*ambitious*
Myself	×	×	×		×	×	×		×	×			×	×
My Ideal	×		×	×	×		×		×				×	×
Father	×	×		×	×	×	×	×	×	×		×	×	×
Mother	×	×		×	×	×		×	×	×		×	×	×
Sister	×	×		×	×	×	×		×	×			×	×
Brother-in-law			×	×	×		×				×	×		×

Figure 1: Data from anorexia treatment (N. Spangenberg and K.E. Wolff)

For example, the patient ("Myself") expresses her opinion that her mother is easily offended, but her sister is not.

The data displayed in Figure 1 consists of three parts:

- The persons Myself, ..., Brother-in-law,
- the 14 atttributes *over-sensitive*, ..., *ambitious*,
- and the crosses that indicate which attributes the patient has assigned to each person.

The anorexia example is typical of the type of data we work with and is therefore cast into a mathematical definition.

Definition 1 A **formal context** $\mathbb{K} := (G, M, I)$ consists of two sets G and M and a relation I between G and M. The elements of G are called the **objects** and the elements of M are called the **attributes** of the formal context. In order to express that an object g is in a relation I with an attribute m, we write $g\ I\ m$ or $(g, m) \in I$ and read it as *"the object g has the attribute m"*. ◊

4 Understanding the Anorexia data

The data in Figure 1 come from a special therapeutic approach to Anorexia nervosa. In this approach, a cause of the eating disorder was seen in the patients' view of their close social environment. The therapy consisted of changing this view through conversation, thereby also influencing the state of the illness.

In each of these interviews, patients were asked to express their views using a standardized procedure in which they were free to choose the attributes themselves. The formal context in Figure 1 shows simplified original data from one such survey.

During the course of therapy, several formal contexts were collected for each patient and then compared. However, crosstables are graphically not very intuitive. The diagrams of the concept lattices, which will now be explained, are much easier to read.

5 Derivation

Let (G, M, I) be some formal context.

Definition 2 For a set $A \subseteq G$ of objects we define

$$A' := \{m \in M \mid g\ I\ m \text{ for all } g \in A\}$$

(the set of all attributes common to the objects in A). Correspondingly, for a set $B \subseteq M$ of attributes we let

$$B' := \{g \in G \mid g\ I\ m \text{ for all } m \in B\}$$

(the set of objects which have all attributes in B). $\Diamond$

Sometimes it is not sufficient to use the same symbol $'$ for the two **derivation operators** $A \mapsto A', A \subseteq G$ and $B \mapsto B', B \subseteq M$. In this case, $A^{\uparrow}$ and $B^{\downarrow}$ may be written to distinguish between them, or, if different formal contexts occur simultaneously, A^I and B^I are used.

Proposition 1 *If (G, M, I) is a formal context, $A, A_1, A_2 \subseteq G$ are sets of objects and $B, B_1, B_2 \subseteq M$ are sets of attributes, then*

1) $A_1 \subseteq A_2 \Rightarrow A_1' \supseteq A_2'$ *1')* $B_1 \subseteq B_2 \Rightarrow B_1' \supseteq B_2'$

2) $A \subseteq A''$ *2')* $B \subseteq B''$

3) $A' = A'''$ *3')* $B' = B'''$

4) $A \subseteq B' \iff B \subseteq A' \iff A \times A \subseteq I.$

The **proofs** are easy.

6 Formal concepts

Definition 3 A **formal concept** of the formal context (G, M, I) is a pair (A, B) of sets with

$$A \subseteq G,\ B \subseteq M,\ A' = B \text{ and } A = B'.$$

We call A the **extent** and B the **intent** of the formal concept (A, B). $\mathfrak{B}(G, M, I)$ denotes the set of all formal concepts of the formal context (G, M, I). $\Diamond$

Visualization: Formal concepts can be illustrated as *maximal full rectangles* in a formal context, i.e., as rectangular parts of the incidence that are filled with crosses and are maximal with respect to this property.

Examples of formal concepts: The method we will now use to construct some examples of formal concepts for Figure 1 will later be replaced by much more efficient techniques. It is based on the following observation, which is easily derived from the definitions combined with Proposition 1:

If $A \subseteq G$ is some set of objects of the formal context, then (A'', A') is a formal concept. If $B \subseteq M$ is a set of attributes, then (B', B'') is a formal concept.

So let $A := \{\text{Myself}, \text{Father}\}$. Then A' is the set of all attributes which the two have in common. From Figure 1 we get

$$A' = \{\text{over-sensitive}, \text{withdrawn}, \text{cordial}, \text{difficult}, \text{attentive}, \\ \text{calm}, \text{apprehensive}, \text{sensitive}, \text{ambitious}\}.$$

Moreover, A'' is the set of those persons that have all attributes in A', thus

$$A'' = \{\text{Myself}, \text{Father}, \text{Sister}\}.$$

In fact, the sister has all attributes which the patient and her father have in common. The sister is also "dutiful", which the patient is not. She therefore does not have *exactly* the common attributes of $A = \{\text{Myself}, \text{Father}\}$, but that is not required for being in A''.

Therefore $A'' = \{\text{Myself}, \text{Father}, \text{Sister}\}$ is the extent and the above listed $A' = \{\text{over-sensitive}, \text{withdrawn}, \ldots, \text{ambitious}\}$ is the intent of a formal concept of (G, M, I).

If we had chosen $A = \{\text{Myself}, \text{Mother}\}$ instead of $A = \{\text{Myself}, \text{Father}\}$ in the above example, then A' would be one attribute smaller, because the mother has all the attributes that the patient shares with her father, except one: she is not attentive. For this choice, we then obtain the concept extent $A'' = \{\text{Myself}, \text{Mother}, \text{Father}, \text{Sister}\}$.

We have thus found two (of a total of 23) formal concepts of the formal context in Figure 1.

7 Style guide



We usually use the letters G, M and I for the sets of objects, attributes and relations. These are the first letters of the German words Gegenstand (object), Merkmal (attribute) and Inzidenz (incidence), and have become established because the first publications on Formal Concept Analysis were mostly written in German. However, these abbreviations have proven useful, and we recommend that readers use them as well.

The frequent use of the word "formal" can be annoying, so it is sometimes omitted. However, in the early days, we simply used "context" and "concept", until it became clear that this was often misunderstood. In today's work on Formal Concept Analysis, the word "formal" therefore appears much more frequently than in the past.

The definition of a formal concept uses the derivation operators, which therefore are also called the **concept forming operators** by some authors.

8 Questions and answers



Q: What exactly are objects, what are attributes, and who determines whether an object has an attribute?

A: We avoid these philosophically difficult questions by allowing, in the mathematical definition, *every* set as a set of objects, *every* set as a set of attributes, and *every* subset of $G \times M$ as an incidence between G and M.

Q: Should the sets G and M be finite, non-empty, and disjoint?

A: No, none of these assumptions are made.

Q: So is every triple (G, M, I) with $I \subseteq G \times M$ a formal context?

A: According to the mathematical definition: yes.

Pragmatically, however, we only call (G, M, I) a formal context if we are interested in the formal concepts of (G, M, I).

Q: Is a formal context the same as a Boolean matrix or a 0-1–matrix?

A: Obviously not!

It is equally obvious that formal contexts can be *represented* by Boolean or 0-1–matrices, as it is often the case in computer programs. However, the intuitive meaning of the data is easily lost when a 0-1 matrix or a table with True and False is used instead of a crosstable.

There are carefully crafted versions that work with graded attributes and fuzzy relations, but we will not go into them here.

9 Concept lattice

The set $\mathfrak{B}(G, M, I)$ of all formal concepts of (G, M, I) can be equipped with an order as follows:

Definition 4 If (A_1, B_1) and (A_2, B_2) are formal concepts of (G, M, I), then (A_1, B_1) is a **subconcept** of (A_2, B_2) if $A_1 \subseteq A_2$ holds (which is equivalent to $B_2 \subseteq B_1$). In this case, (A_2, B_2) is a **superconcept** of (A_1, B_1), and we write

$$(A_1, B_1) \leq (A_2, B_2).$$

The relation $\leq$ is called the **hierarchical order** (or simply **order**) of the concepts. The ordered set of all formal concepts of (G, M, I) is denoted by $\underline{\mathfrak{B}}(G, M, I)$ and is called the **concept lattice** of the formal context (G, M, I). $\Diamond$

10 A smaller example: stirrers

The data in Figure 2 is from our local hardware store which, among many other tools, sells five different types of stirrers. The formal context shows the materials for which the stirrers are suitable. The data has been simplified, the original data will be discussed later (in Section 24).

Stirrers	*paints*	*bitumen*	*grouting compounds*	*slurry*	*plaster*	*leveling compounds*	*concrete*
paint stirrer	×						
paddle stirrer	×	×		×			
disc stirrer	×	×	×	×		×	
helical stirrer					×	×	
mortar mixer				×	×	×	×



Figure 2: Which stirrer for which material?

Here are three (of the ten) formal concepts in ascending order:

({mortar mixer}, {*slurry*, *plaster*, *concrete*, *leveling comp.*}) $\leq$
({mortar mixer, disc stirrer}, {*slurry*, *concrete*}) $\leq$
({mortar mixer, disc stirrer, paddle stirrer}, {*slurry*})

11 Concept lattice diagrams

Small concept lattices can be clearly represented using diagrams. This method is effective for up to around 40 concepts, but more advanced representations are usually required for larger lattices. These types of diagram are however quite different from the more familiar bar or pie charts. With a little practice, though, they can be extremely informative. We explain the reading rules using the example in Figure 3, which contains a diagram for the stirrers concept lattice.

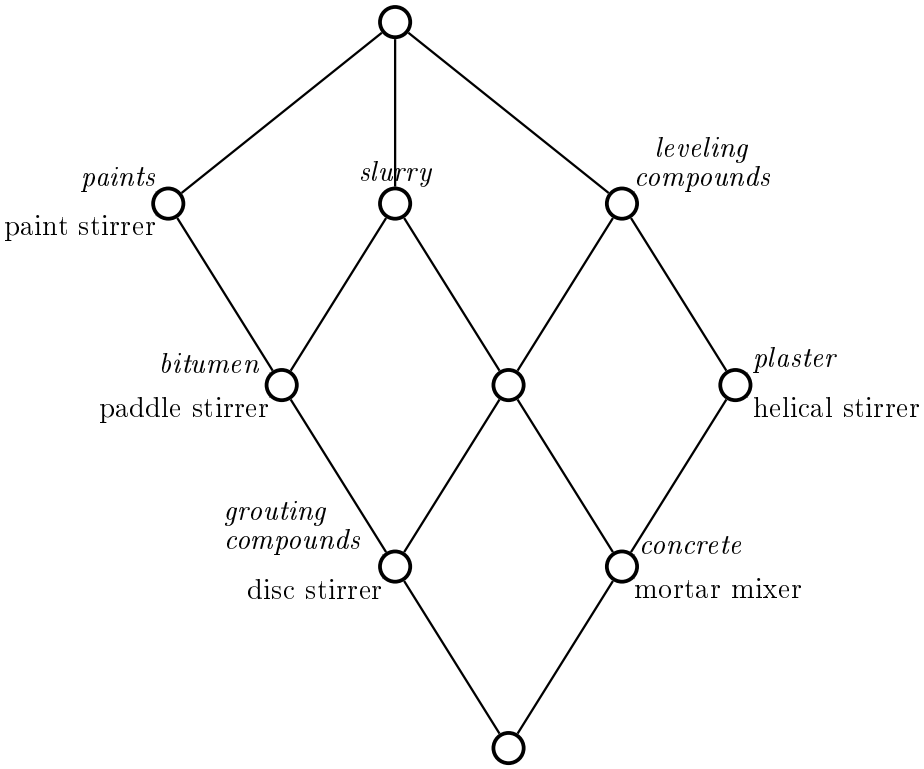


Figure 3: A diagram for the concept lattice corresponding to Figure 2

The ten small circles correspond to the ten formal concepts of the formal context from Figure 2. They are arranged in the figure so that hierarchically larger ones are drawn higher up. More precisely, a formal concept is a subconcept of a second one if and only if it is possible to move from its circle to the circle of the second one by following the edges in ascending order.

How can you read the concept extent and intent from the diagram? We explain this using the example of the completely unlabeled circle, which is located approximately in the middle. This circle represents the formal concept

$$(\{\text{disc stirrer}, \text{mortar mixer}\}, \{\textit{slurry}, \textit{leveling compounds}\}).$$

This can be seen in the diagram as follows: Starting from the circle, follow *all* paths leading downwards. The concept extent consists of the object names along such paths. For the concept intent, follow all upward paths from the circle and collect all attribute names (which here are written in *italic* font). Therefore, the concept extent is

$$\{\text{disc stirrer}, \text{mortar mixer}\}$$

and its intent is

$$\{\textit{slurry}, \textit{leveling compounds}\}.$$

More examples for reading Figure 3

What are extent and intent of the formal concept which corresponds to the circle at the top right labeled *leveling compounds*? The intent actually has only one element, namely $\{\textit{leveling compounds}\}$. The concept extent is three-element:

$$\{\text{disc stirrer}, \text{mortar mixer}, \text{helical stirrer}\}.$$

Which formal concept corresponds to the unlabeled circle at the top of the diagram? The concept intent is empty, because no attribute name can be reached along an upward path from that circle. The concept extent consists of all five objects, because *every* object name can be reached following downward paths from the top circle. The formal concept thus is

$$(\{\text{paint stirrer}, \text{paddle stirrer}, \text{disc stirrer}, \text{mortar mixer}, \text{helical stirrer}\}, \emptyset).$$

Similarly, the formal concept corresponding to the bottom circle has an empty extent and the set of *all* attributes as intent.

An empty concept intent or extent is in fact completely compatible with Definition 1. The associated extent is always the set G of all objects, or the set M of all attributes of the formal context, respectively.

However, this does not mean that these border concepts must always be unlabeled, as in this example. In Figure 4 below, the top element is labeled with two attribute names. These correspond to the two full columns in the formal context in Figure 1.

12 A diagram for the Anorexia data

Now we are able to draw and interpret a diagram for the concept lattice of the Anorexia context (Figure 1), see Figure 4. Such diagrams were actually used by the therapists in the study.

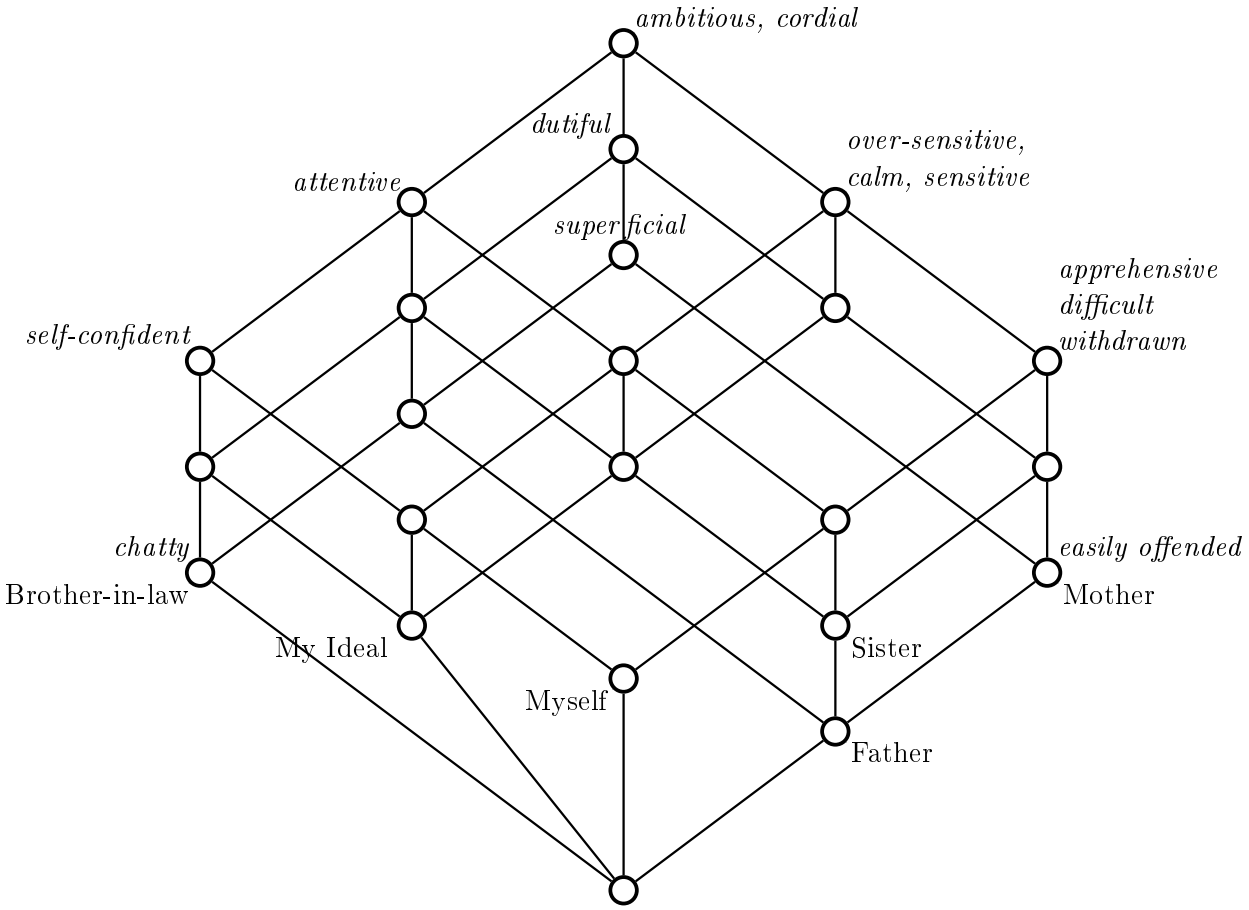


Figure 4: A concept lattice diagram for Figure 1

Consider, for example, the two unlabeled circles in the middle. The higher one, just beneath the circle which is labeled "*superficial*", has as its extent

$$\{\text{Father, Sister, Myself, My ideal}\}.$$

The intent is

$$\{\textit{ambitious, cordial, over-senditive, clam, sensitive, attentive}\}.$$

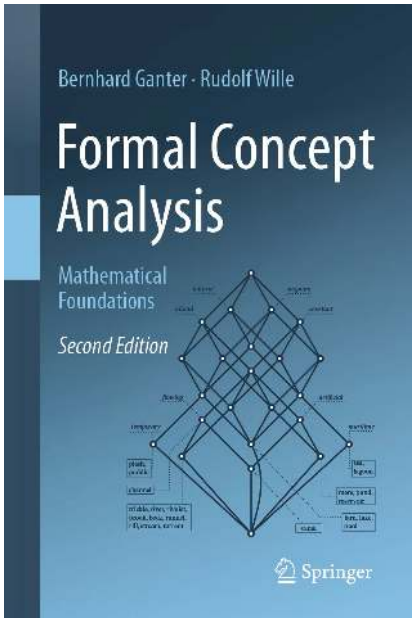
The concept just below that one exclodes "Myself" from the extent, but adds "*dutiful*" to the intent. Unlike everyone else, she does not describe herself as "*dutiful*".

13 Two angry questions answered by an advertisement



Q: Why all this effort with Q&A, Style guide, etc.? Can't you just provide concise and precise mathematical information, with definitions, theorems, and proofs?

A: Well, that's been around since the last century. Our book "Formal Concept Analysis – Mathematical Foundations", which was written in the 1990s, is now in its second edition. It contains 120 definitions, 72 theorems, and 147 propositions, with proofs. The book is cited unusually often and includes 430 references itself. Of course we recommend it. This text is based on it.



So if you are brave enough to read such a book, then this is the best source we can offer. However, it is doubtful whether this is the best approach for beginners. Even professionals occasionally reinvent things that have long been covered in detail in the literature. However, we do not see Formal Concept Analysis as merely a branch of mathematics (or computer science), but believe that it has significance beyond this. In order to be understood outside the circle of experts, it requires a uniform and meaningful technical language, good graphics, and a convincing justification of its approach. This `fca_math` enlightener attempts to emphasize these aspects. Whether readers will like it remains to be seen.

Q: Couldn't you at least leave out the stuff about Anorexia therapy and hardware store stirrers et cetera, call the objects a, b, c, ... and the attributes u, v, w, ... so that it looks more like normal mathematics?

A: The basic mathematical construction of Formal Concept Analysis, namely the construction of complete lattices from arbitrary binary relations, can already be found in G. Birkhoff's famous book "Lattice Theory", which was published in the first half of the 20th century. However, little attention was paid to this technique. It was only when Formal Concept Analysis demonstrated its practical significance in a large number of real-world examples that the idea was taken up in thousands of publications. That is why we have made it a habit to use data taken from real life rather than "fake", self-invented data when providing examples of formal contexts.

14 Python programs for Formal Concept Analysis

Many authors have contributed software to Formal Concept Analysis, recently often in the Python programming language. Our version is called `fca_math`. It is nothing special, except that it is based on the book discussed in Section 13. There is a package manual and there are source code listings, but these are not needed to get started. It is assumed that Python3 is installed on your computer, that the `fca_math` package is located in a suitable folder, and that you know how to open a Python shell. It greets you with a prompt:

```
>>>
```

Answer by telling that you plan using `fca_math`:

```
>>> from fca_math import *
```

To do something, we need a formal context. Fortunately, the package has some predefined ones which can be loaded. The 'Stirrers' context is among them:

```
>>> kk = Context.read('Stirrers')
```

When you press 'Enter', all you get is another prompt:

```
>>> kk = Context.read('Stirrers')
>>>
```

In fact, the formal context with the name 'Stirrers' was loaded and assigned to a variable named `kk`. Really? Let us have a look:

```
>>> print(kk)
```

The result is somewhat surprising:

```
\begin{cxt} %  5 objects, 7 attributes
\cxtName{Stirrers}
\atr{paints} % 0
\atr{bitumen} % 1
\atr{grouting compounds} % 2
\atr{slurry} % 3
\atr{plaster} % 4
\atr{leveling compounds} % 5
\atr{concrete} % 6
\obj{x.....}{paint stirrer}  % 0
\obj{xx.x...}{paddle stirrer}  % 1
\obj{xxxx.x.}{disc stirrer}  % 2
\obj{....xx.}{helical stirrer}  % 3
\obj{...xxxx}{mortar mixer}  % 4
\end{cxt}
```

What you get is LaTeX-code for typesetting the crosstable using the LaTeX `fca.sty` package. This code was used for typing Figure 2. To obtain only the crosstable, enter `kk.print_incidence()` instead.

15 First computations

After reading in the formal context `Stirrers.cxt` with the command

```
>>> kk = Context.read('Stirrers')
```

and thereby assigning it to the variable `kk`, we can query the first results. For example, we can ask for the number of formal concepts:

```
>>> kk.number_of_concepts()
10
```

We can also display these concepts as pairs of extent and intent, in accordance with their definitions. The function `twenty_concepts(start)` lists up to twenty formal concepts in decending order. By default, `start = 0`.

```
>>> kk.twenty_concepts()

0 : ( {0, 1, 2, 3, 4} , set() )
1 : ( {2, 3, 4} , {5} )
2 : ( {3, 4} , {4, 5} )
3 : ( {1, 2, 4} , {3} )
4 : ( {2, 4} , {3, 5} )
5 : ( {4} , {3, 4, 5, 6} )
6 : ( {0, 1, 2} , {0} )
7 : ( {1, 2} , {0, 1, 3} )
8 : ( {2} , {0, 1, 2, 3, 5} )
9 : ( set() , {0, 1, 2, 3, 4, 5, 6} ) .
```

Quite useful is the successor list, which in this case is invoked as follows:

```
>>> kk.successor_list()
0 : --
1 : 0
2 : 1
3 : 0
4 : 3 1
5 : 4 2
6 : 0
7 : 6 3
8 : 7 4
9 : 8 5  .
```

The following can be read: The formal concept 0 has no successors, it is the top formal concept. The concepts 1, 3 and 6 only have the successor 0. 2 has 1 as its only successor, 4 has 3 and 1 as successors, 5 has 4 and 2, etc. This kind of information is very helpful when drawing a diagram of a concept lattice by hand.

16 Drawing a concept lattice diagram by hand

We will work with computer-generated diagrams later on. But for starters, we recommend drawing small diagrams by hand. So let's forget for a moment that we already know a diagram for the "stirrers" lattice (Figure 3) and draw a new one. This is easy if you use the lists discussed on the previous page. The step-by-step procedure is illustrated in Figures a to e.

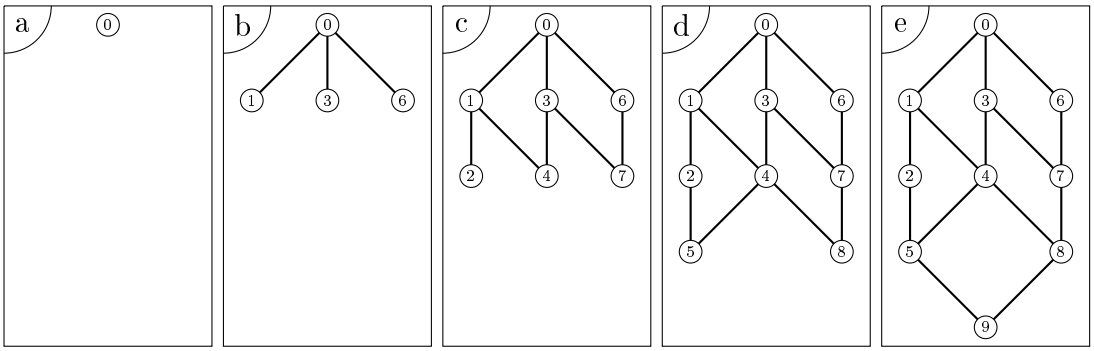
a The highest concept is the one with the number 0. We draw a small circle at the top for this.

b The concepts numbered 1, 3, and 6 have 0 as their successors, so they are lower neighbors of the top concept and are drawn accordingly.

c Formal concept number 2 has only number 1 as its successor, 4 has 1 and 3, and 7 has 3 and 6 as its successors.

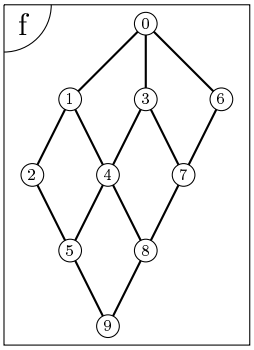
d Conept 5 is below 2 and 4, concept 8 is below 4 and 7, and

e concept number 9 is below 5 and 8 and is the bottom concept.



With Figure e, we have already achieved a valid lattice diagram. However, two steps are still missing.

f You can move the small circles around as you like, as long as you don't change the hierarchical order. Usually, after the initial drawing, several improvement steps are necessary to make the diagram "nice" and readable. In our example, this is not really necessary, but Figure f is a little better structured than Figure e.



The final step is to enter the labels and remove the small numbers from the circles. This makes the diagram readable without referring to the tables we calculated on the previous page. However, we need to request another table from Python for this, telling us which of the small circles the labels must be attached to. This will be explained on the next page.

17 Where the labels go

The diagrams in Figures 3 and 4 are labeled, but this does not mean that the extent and intent at each concept circle are specified. That would overload the figures to the point of illegibility. Instead, each object and each attribute name is entered exactly once as a label. We have already shown in Section 12 that even with this *reduced* labeling, every concept extent and intent can be read. But where do you write the labels so that it works?

The answer is that the name of an object g is always written at the **object concept**

$$\gamma g := (\{g\}'', \{g\}')$$

and the name of the attribute m is written at the **attribute concept**[1]

$$\mu m := (\{m\}', \{m\}'').$$

For example the paddle stirrer is object number 1 in the stirrers context (see Section 15. Therefore "paddle stirrer" must be written slightly beneath the circle for the formal concept $(1'', 1')$. This can be computed as follows:

```
>>> one = {1}
>>> oneprime = kk.make_intent(one)
>>> onedoubleprime = kk.make_extent(oneprime)
>>> print('(',onedoubleprime,',',oneprime,')'
( {1, 2} , {0, 1, 3} )
```

The outcome is just number 7 of the formal concept list on Page 13, so the label "paddle stirrer" goes slightly beneath the circle labeled "7".

We could do this for all objects and all attributes, but we don't have to because there is a Python function available for this purpose:

```
>>> kk.label_assignment()
object : concept | attribute : concept
    0  :  6              0  :  6
    1  :  7              1  :  7
    2  :  8              2  :  8
    3  :  2              3  :  3
    4  :  5              4  :  2
                         5  :  1
                         6  :  5
```

If you enter the labels according to these assignments and remove the concept numbers from the circles, you get exactly the diagram from Figure 3, except that it is mirrored.

[1] For single-element sets, it is convenient to omit the set brackets and write (g'', g') or (m', m'')

18 Style guide (diagrams)



On the one hand, there are no fixed rules for the graphical representation of concept lattices. The founder of this field, Rudolf Wille, experimented with alternative forms on several occasions.

On the other hand, it must be borne in mind that most people are unfamiliar with such diagrams. They are used to pie charts, bar charts, or function diagrams, which are read differently. The reading rules must be explained and practiced, especially the reduced labeling, otherwise the diagrams will easily be misinterpreted. This argues in favor of using uniform graphics and shows that it makes sense to invest some effort in creating particularly good diagrams.

But even well-designed lattice diagrams rarely work for more than, say, forty formal concepts. This corresponds to the way we deal with human concepts. I cannot recall any representations of human conceptual systems that clearly show a large number of concepts in their full hierarchy. This problem is usually solved by enforcing a tree structure (and drastically restricting the set of attributes). In Formal Concept Analysis, new methods have been developed that allow even larger sets of formal concepts to be visualized. This will be discussed later.

We recommend not writing any text inside the circles representing concepts, except during the design phase (see above). Colors, shadows, etc. should only be used if they have a clearly understandable conceptual meaning.

We do not know exactly what makes a diagram easy to read. A layered structure often helps, but is not always ideal. Experience shows that parallelograms contribute to readability.

Such diagrams are often referred to as "Hasse diagrams" when they represent lattices or other ordered sets. We consider this to be a misnomer; the number theorist Hasse contributed little to this field. We prefer to refer to them as line diagrams, order diagrams, or lattice diagrams.

19 Exercises

Draw concept lattice diagrams (with labels) for these three formal contexts:

	natural	*flowing*	*artificial*	*stagnant*
river	×	×		
canal		×	×	
pond			×	×
sea	×			×

	natural	*stagnant*	*constant*	*artificial*
puddle	×	×		
river	×		×	
lake	×	×	×	
canal			×	×
pond		×	×	×

	inland	*flowing*	*temporary*	*maritime*
pond	×			
trickle	×	×		
puddle	×		×	
lagoon				×

20 Computing with extents and intents

Each formal concept (U, V) has concept *extent* U and concept *intent* V. These were defined in Section 6 using the *derivation operators* introduced in Section 5. Of course there are also `Python` methods for these:

- Extents and intents
  - `B = kk.make_intent(A)`
  - `A = kk.make_extent(B)`
  - `CA = kk.close_extent(A)`
  - `CB = kk.close_intent(B)`
  - `FE = kk.first_extent()`
  - `FI = kk.first_intent()`
  - `NA = kk.next_extent(A)`
  - `NB = kk.next_intent(B)`
  - `ul = kk.uplist()`

Figure 5: Python methods for concept extents and intents

When A is a set of objects of a formal context `kk`, then `kk.make_intent(`A`)` returns the set A', and `kk.close_extent(A)` produces the set A'', etc.

The `first_` and `next_` functions can be used to generate all concepts one after the other. F = `kk.first_intent()` returns the intent of the largest formal concept of `kk`. Continuing with N=`kk.next_intent(`F`)` yields the intent of a lower neighboring concept. And repeated invocation of N =`kk.next_intent(`N`)` returns all concept intents in descending order, one after the other.

The latter can also be achieved by calling `ul = kk.uplist()` once. This returns a list `ul` of triples of the form (extent,intent,lower neighbors), sorted in descending order of intents, which iterates through all formal concepts of `kk`.

Let us take the middle context from Exercise 19 as an example and give it the variable name kk. Then

`>>> kk.make_intent({'lake','pond'})`
gives the result `{1,2}` and
`>>> kk.make_extent({1,2})` yields
`{2,4}`, which stands for `{'lake','pond'}`[2].
`>>> kk.close_intent({'lake','pond'})`
gives the same result.

[2]Some functions are quite tolerant with regard to the input format of sets, but only in interpreter mode. Internally, indices are always used.

21 Complete Lattices and the Basic Theorem

An **ordered set** is a set P together with a *reflexive*, *transitive*, and *antisymmetric* relation $\leq$ on P. In this case, $\leq$ is called an **order (relation)**.

Definition 5 An element $p \in P$ in an ordered set $(P, \leq)$ is a **lower bound** of a subset $S \subseteq P$ if $p \leq s$ holds for every $s \in S$. An element which is greater or equal to every element of S is an **upper bound** of S.

It may happen that the set of all lower bounds of $S \subseteq P$ has a greatest element. Then this is called the **infimum** of S and is denoted $\bigwedge S$.

Dually, a least upper bound of S is called the **supremum** of S and is denoted $\bigvee S$. ◊

Supremum or infimum do not have to exist, but if they do, they are unique. If *every* element of P is the infimum of some subset os S, then S is **infimum-dense** in P. Dually, S is **supremum-dense** if every element of P is the supremum of sone subset of S.

Definition 6 A **complete lattice** is an ordered set $(L, \leq)$ in which supremum and infimum exist for every subset $S \subseteq L$. ◊

Theorem 1 (The Basic Theorem on Concept Lattices) *The concept lattice* $\underline{\mathfrak{B}}(G, M, I)$ *is a complete lattice in which infimum and supremum are given by*

$$\bigwedge_{t \in T} (A_t, B_t) = \Big(\bigcap_{t \in T} A_t \,,\, (\bigcup_{t \in T} B_t)'' \Big)$$

$$\bigvee_{t \in T} (A_t, B_t) = \Big((\bigcup_{t \in T} A_t)'' \,,\, \bigcap_{t \in T} B_t \Big).$$

A complete lattice $\boldsymbol{L} := (L, \leq)$ *is isomorphic to* $\underline{\mathfrak{B}}(G, M, I)$ *if and only if there are mappings* $\tilde{\gamma} : G \to L$ *and* $\tilde{\mu} : M \to L$ *such that* $\tilde{\gamma}(G)$ *is supremum-dense in* $\boldsymbol{L}$, $\tilde{\mu}(M)$ *is infimum-dense in* $\boldsymbol{L}$, *and* $g\ I\ m$ *is equivalent to* $\tilde{\gamma} g \leq \tilde{\mu} m$ *for all* $g \in G$ *and all* $m \in M$. *In particular,* $\boldsymbol{L} \cong \underline{\mathfrak{B}}(L, L, \leq)$.

The theorem looks more complicated than it is. The first statement is that every concept lattice is a complete lattice, i.e., for every set of formal concepts in a context, the supremum and the infimum exist. And it doesn't matter whether the set is finite or infinite. Even the empty set has a supremum and an infimum. And how do you find the infimum and supremum? The main formula provides the answer: to find the infimum, take the intersection of the sets, and for the supremum, take the intersection of the contents. This uniquely determines the respective formal concept. The rest clarifies which complete lattices are (isomorphic to) concept lattices: all of them.

22 Style guide (Basic Theorem)



Ordered sets are often referred to as "partially ordered sets" to indicate that **incomparable** pairs may occur, i.e., elements a and b for which neither $a \leq b$ nor $b \leq a$ holds. We omit the word "partially" and instead refer to ordered sets that have no incomparable pairs as "linearly ordered" sets or as "chains".



Supremum and infimum correspond to the *least common multiple* and the *greatest common divisor* of natural numbers. The natural numbers, ordered by divisibility, actually form a lattice (you have to add ∞ to make it a complete lattice), with *gcd* and *lcm* as infimum and supremum. By analogy, infimum and supremum of formal concepts can also be referred to as the "greatest common subconcept" and "least common superconcept," respectively.

Theorem 1 is the **Basic Theorem on Concept Lattices**. It is *not* the "Fundamental Theorem" or the "Main Theorem" of Formal Concept Analysis. What is the difference between basic and fundamental here? Theorem 1 is fundamental in that it conclusively characterizes the construction of the concept lattice from a formal context: The result is always a complete lattice, and every complete lattice is isomorphic to a concept lattice. This result is used repeatedly in Formal Concept Analysis.

On the other hand, Theorem 1 is neither particularly deep nor difficult to prove. That is why it is not called "fundamental," but rather "basic". Formal Concept Analysis has more difficult results to offer, so calling it the "main theorem" would be misleading.

Finally, it is not the only "basic theorem" in Formal Concept Analysis. There are other mathematical structures that can be derived from formal contexts, and there are fundamental characterizations for these as well. That is why it is called the basic theorem "on concept lattices".

This suggests a strategy for formulating future developments of the theory. Let us take a fictional example and assume that you have invented a new type of conceptual unit that you call "bubble concept". And you show that bubble concepts can be treated mathematically, that they form a "bubble concept algebra". Then you should formulate and prove a "basic theorem on bubble concept algebras" that characterizes the mathematical properties of such algebras so precisely that every algebra with these properties is (isomorphic to) a bubble concept algebra.

The first part of the Basic Theorem, the construction of a complete lattice from a relation, can already be found in G. Birkhoff's famous book "Lattice Theory", albeit in a purely mathematical formulation without reference to concepts. However, this construction received little attention. The original contribution of the Formal Concept Analysis therefore lies not in this construction, but in the conceptual interpretation, which was then consistently expanded.

And its strength also lies in the extensive mathematical theory of lattice theory, from which it can draw. What a wonderful source of mathematical results!

23 Questions and answers (Basic Theorem)



Q: So there is a formal context for every complete lattice?

A: The context is definitely not unique. This can be seen from the examples above: The anorexia concept lattice in Figure 4 has 23 formal concepts, and thus the formal context $(L, L, \leq)$ specified by the Basic Theorem has 23 objects and 23 attributes and is thus much larger than the one in Figure 1.

Q: If it is not unique, is there at least a canonical one?

A: The context $(L, L, \leq)$ specified by the Basic Theorem can be called canonical, but it is far too large and cumbersome for many purposes. It would be desirable to always use the smallest possible context. However, additional conditions on a complete lattice are required for such a smallest representing context to exist. A sufficient condition is finiteness, a more general one is called "double foundedness".

Q: What is the difference between a lattice and a complete lattice?

A: For a complete lattice it is required that the supremum $\bigvee S$ and the infimum $\bigwedge S$ exist for every subset S. For a lattice this is only required for sets with two elements, i.e., with $|S| = 2$. Every finite lattice is complete, except for te empty one.

Q: So every complete lattice is a concept lattice?

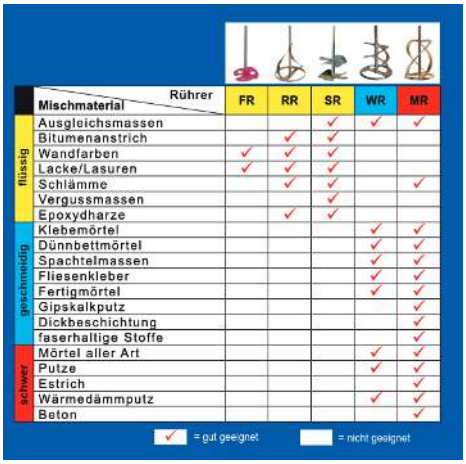
A: That is not what the Basic Theorem says, and it is obviously not true. Every complete lattice is *isomorphic to* a concept lattice, but if the elements aren't formal concepts, it certainly is not a concept lattice. "Isomorphic" means "the same up to a renaming of the elements".

Q: The mappings $\tilde{\gamma}$ and $\tilde{\mu}$ have unusual names. Why?

A: In Section 17, mappings γ and μ were introduced as the "label mappings": γ maps the object g to the object concept $\gamma g := (g'', g')$, and μ is defined accordingly for attributes. The mappings in the Basic Theorem are not exactly the same as γ and μ, but they are close, since they are the label mappings, followed by the isomorphism. That is why they need different names, but names that are reminiscent of γ and μ.

24 Clarifying a formal context

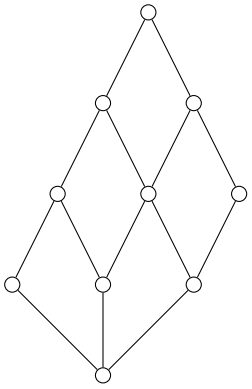
The formal context of stirrers in Figure 2 is not the original, see the image below.[3]



	Mischmaterial \ Rührer	FR	RR	SR	WR	MR
flüssig	Ausgleichsmassen			✓	✓	✓
	Bitumenanstrich		✓	✓		
	Wandfarben	✓	✓	✓		
	Lacke/Lasuren	✓	✓	✓		
	Schlämme		✓	✓		✓
	Vergussmassen			✓		
	Epoxydharze		✓	✓		
geschmeidig	Klebemörtel				✓	✓
	Dünnbettmörtel				✓	✓
	Spachtelmassen				✓	✓
	Fliesenkleber				✓	✓
	Fertigmörtel				✓	✓
	Gipskalkputz					✓
	Dickbeschichtung					✓
	faserhaltige Stoffe					✓
schwer	Mörtel aller Art				✓	✓
	Putze				✓	✓
	Estrich					✓
	Wärmedämmputz				✓	✓
	Beton					✓

✓ = gut geeignet = nicht geeignet

Three differences stand out: a) The text is in German, b) the table is transposed, i.e., rows and columns are interchanged, and c) the list of stirrable materials is much longer: the table has 20 rows, whereas the formal context in Figure 2 has only seven attributes. But if you calculate the concept lattice of this table and draw it (without labels), you get the diagram shown below on the left. It is exactly the same as in Figure 3, only upside down. How can this be explained?



The diagram is upside down because objects and attributes were swapped when the table was transposed. This process is called **dualizing**, and we will revisit it in Section **??**.

The table has many rows where the same columns are checked. Only seven of the rows are different (apart from the names), and these correspond exactly to the seven columns in Figure 2. This process of grouping together identical rows of a formal context (and identical columns accordingly) is called **clarifying** the context. Clarification does not change the lattice structure, it only changes the labels.

However, it must be said that in Figures 2 and 3, we have simplified the labeling more than necessary. All information is retained if, when merging identical rows or columns of a formal context, the labels are also merged into a list. Instead, we have taken one name each time and omitted the rest to keep the example simple. If the original data is used as a (dual) formal context, then 15 labels are added in Figure 3, with the diagram remaining unchanged.

The formal mathematical definition of the clarification of (G, M, I) is

$$(G, M, I)^\circ := (G/\ker\gamma, M/\ker\mu, I^\circ),$$

with $(g, h) \in \ker\gamma :\iff \gamma\, g = \gamma\, h$ (similarly for $\ker\mu$) and

$$([g]\ker\gamma, [m]\ker\mu) \in I^\circ :\iff g\ I\ m.$$

There is a Python command `kk.clarify()` for that in the package.

[3] from the 2025 catalogue of Haromac company

25 Reducing a formal context

The context in Figure 2 (p. 7) can be reduced even further without changing the structure of the lattice. The attributes *bitumen*, *grouting compounds* and *concrete* can simply be omitted without changing the number or the order of the formal concepts. Only the labeling changes in Figure 3, with these three labels being omitted.

To explain, let us start with the attribute *concrete* as an example. The mortar mixer is the only one (of the five mentioned here) that is suitable for mixing concrete. The attribute extent of *concrete* is therefore the single-element set {mortar mixer}. But this concept extent can also be specified without using this attribute: The mortar mixer is the also only one that can handle both *slurry* and *plaster*. The attribute *concrete* is dispensable, since it can be expressed by the attribute combination "*slurry* and *plaster*". Similarly for *bitumen*: We find that

$$\mathit{bitumen}' = \{\text{paddle stirrer}, \text{disc stirrer}\},$$

but we also have that

$$\{\mathit{paints},\ \mathit{slurry}\}' = \{\text{paddle stirrer}, \text{disc stirrer}\}.$$

Definition 7 An attribute m of a clarified formal context (G, M, I) is **reducible** iff there is a set $X \subseteq M$ of attributes with $m \notin X$ but $m' = X'$. Dually, $g \in G$ is reducible iff $g' = X'$ for some $X \subseteq G$ with $g \notin X$. ◊

Definition 8 An element x of a complete lattice $\boldsymbol{L}$ is **$\bigvee$-irreducible**[4] if x is not a supremum of other elements, and **$\bigwedge$-irreducible**[5] if it is not an infimum of other elements. The set of $\bigvee$-irreducible elements of denoted $\mathbf{J}(\boldsymbol{L})$, that of $\bigwedge$-irreducible ones by $\mathbf{M}(\boldsymbol{L})$. ◊

Proposition 2 *An element of a finite lattice is $\bigvee$-irreducible iff it has a unique lower neighbor, and $\bigwedge$-irreducible iff it has a unique upper neighbor.*

There is a numbered diagram of the stirrers-lattice on page 14. The elements 1, 2, 3, and 6 are the $\bigwedge$-irreducible ones, 2, 5, 6, 7, and 8 the $\bigvee$-irreducibles.

Proposition 3 *Every $\bigvee$-irreducible element of a concept lattice is an object concept, and every $\bigwedge$-irreducible one is an attribute concept. An object is irreducible iff its object concept is $\bigvee$-irreducible, an attribute is irreducible iff its attribute concept is $\bigwedge$-irreducible.*

The `fca_math` command for reducing is `kk.reduce()`.

[4] "completely join-irreducible"
[5] "completely meet-irreducible"

26 Questions and answers (clarifying and reducing)



Q: If object or attribute names are combined into lists during clarification, does this not make the diagram unreadable?

A: Not always. See, e.g., Figure 4.

Q: When does one use "supremum" and "infimum", when "join" and "meet"?

A: Supremum and join are synonyms, as are infimum and meet. Supremum and infimum come from analysis, while join and meet come from geometry. Which ones you use is a matter of taste. I use both.

Q: Can every formal context be reduced?

A: Well, every finite clarified context can be reduced. The result is uniquely determined and has the same concept lattice, except for a few labels.

With infinite sets, things can go wrong. The real interval $[0, 1]$ with the usual order $\leq$ is a complete lattice, and according to the basc theorem, $([0, 1], [0, 1], \leq)$ is a formal context, the concept lattice of which is isomorphic to $([0, 1], \leq)$. This formal context has no irreducible elements at all. If you reduce it, it disappears.

Q: Should finite contexts always be reduced?

A: Reducing makes the context compact and facilitates calculations. However, omitting the labels of reducible elements in the diagram results in a loss of information.

Q: Is it possible to reduce without clarifying?

A: Can be defined, but is rarely used. In the definition of a reducible attribute, replace "$m \notin X$ but $m' = X'$ " by "$m' \notin \{x' \mid x \in X\}$ but $m' = X'$ ", and similarly for objects.

Q: You use the variable name `kk` for formal contexts. Is this important?

A: No. You can use any variable name that is valid in `Python`. In mathematical text, we like to use the symbol $\mathbb{K}$, and `kk` is similar.

27 Finger exercises: Fun with tiny lattices

- There are exactly nine reduced formal contexts with three objects, apart from isomorphisms. Try to draw and label the (small) concept lattices without computer help.

River discharge	$\geq 15\,000\ m^3/s$	$\geq 200\,000\ m^3/s$	$\geq 250\,000\ m^3/s$
Amazonas	×	×	
Mississippi	×		
Ganges			

Primes < 6	*odd*	$\equiv 1 \pmod 4$	$\equiv 3 \pmod 4$
2			
3	×		×
5	×	×	

Quadrangles	*equal side lengths*	*≥ 1 right angle*	*4 right angles*
◇	×		
□		×	×
▱		×	

Free from	*lactose free*	*vegetarian*	*vegan*
egg	×	×	
beef	×		
milk		×	

Animals	*aquatic*	*mammal*	*predator*
lynx		×	×
shark	×		×
manatee	×	×	

States	*african*	*asian*	*european*
Cameroon	×		
Japan		×	
Italy			×

Tempi	*slow*	*moderate*	*fast*
Moderato		×	×
Andante	×	×	
Adagio	×		

Comparison	≤ 1	≤ 2	≥ 2	≥ 3
1	×	×		
2		×	×	
3			×	×

3–4–5	*odd*	$\equiv 1 \pmod 4$	$\equiv 3 \pmod 4$	*even*
3	×		×	
4				×
5	×	×		

- How many reducible objects and attributes are there in the Anorexia data (Figures 1 and 4)?

- The set of all divisors of the natural number 60 is a lattice $\boldsymbol{L}$, when ordered by divisibility. Determine the sets $\mathbf{J}(\boldsymbol{L})$, $\mathbf{M}(\boldsymbol{L})$, and describe extents and intents of the formal concepts of $\underline{\mathfrak{B}}(\mathbf{J}(\boldsymbol{L}), \mathbf{M}(\boldsymbol{L}), \leq)$.

28 Standard contexts available for `fca_math`

The `fca_math` package provides many predefined formal contexts. Standard contexts that are more mathematical in nature (and often are very simple) are referred to in Formal Concept Analysis as **conceptual scales** and often have fixed names. The exact definitions can be found in the book mentioned above.

`Scale` is a subclass of `Context`.

- `Context(g,m)`
- `Scale.nominal(n)`
- `Scale.contranominal(n)`
- `Scale.ordinal(n)`
- `Scale.interordinal(n)`
- `Scale.integer_partitions(n)`
- `Scale.set_partitions(n)`
- `Scale.noncrossing(n)`
- `Scale.tamari(n)`
- `Scale.permutations(n)`
- `Scale.additively_saturated(n)`
- `Scale.pg2(d)`
- `Scale.dichotomic()`
- `Scale.biordinal(m,n)`
- `Scale.contraordinal(rel)`
- `Scale.max_antichains(rel)`
- `Scale.general_interordinal(rel)`
- `Scale.convex_ordinal(rel)`

Above, d, g, m, and n are natural numbers, `rel` stands for a relation, often an order, and will be discussed later on. Typing

```
>>> kk = Context(4,6)
```

assigns a formal context with four objects (named 0, 1, 2, 3), six attributes (named 0, 1, ..., 5), and an empty incidence relation to the variable `kk`.

29 Editing a formal context

So you have typed
`>>> kk = Context(3,3)`
and created a 3×3 context with empty incidence. What now? We need methods for modifying the incidence and changing the object and attribute names.

These will now be explained, but first I must ask for your patience. The methods presented first are somewhat clumsy. They will be replaced by easier ones, but I recommend not skipping them.

The basic method for changing the incidence relation of a context `kk` is
`>>> kk.make_incident(g,m)`.
So if you type
`>>> kk.make_incident(1,2)`
the incidence is no longer empty, but contains the pair $(1, 2)$.

Using this command repeatedly, you can generate the "Animals" context from the finger exercises (Section 27 above), more precisely: its incidence.

You can also enter the names of objects and attributes, as well as context names. Here's how:[6]

```
>>> kk.gname[0] = 'lynx'    ...
>>> kk.mname[2] = 'predator'    ...
>>> kk.name = 'Animals'
```

And from then on, you can use these names. Typing
`>>> kk.make_incident('lynx','predator')`
gives lynx the attribute *predator*.

It couldn't be easier. However, it is important to note that such names are strings. This means that quotation marks must always be used, otherwise an error message will appear.

Some people (mainly me) find this annoying. Therefore, in `fca_math`, all objects and attributes have *two* names: the assigned string and a number from the set $\{0, 1, 2, \ldots\}$. They can even be used interchangeably.

But be careful! You have to keep numbers and strings separate. 3 is not the same as '3'. It is not unusual for an object with the number 0 to have the name '3'. This is the case, for example, with the 3–4–5 context in the finger exercises above. For that formal context, the commands
`>>> kk.make_incident(0,'odd')` and
`>>> kk.make_incident('3',0)`
have the same effect.

It must be admitted that the use of numbers as names does not exactly reflect the mathematical definition of a formal context, because they give the objects and attributes an *order*. However, as long as this order is not explicitly used, this does not cause any problems.

[6] Should be changed to setter methods, to avoid repeated names.

30 Make context manipulation more convenient

The Anorexia context (p. 3) has 58 crosses. You probably don't want to type the `make_incident(g,m)` command 58 times to create this context. We need simpler methods! And yes, there are.

We do not offer a context editor or a GUI here. The `fca_math` package is intended for use with the command line. More convenient solutions are very welcome. But even so, the input can be simplified considerably.

```
>>> kk.set_intent(g,intent)
```

allows you to input a formal context line by line. Here, `g` must be an object name (i.e., a number or a string), and `intent` must describe a subset of the set of attributes. For example, when typing in the last row of the "Animals" context (p. 24), you may type one of

```
>>> kk.set_intent('manatee',{0,1})
```
or
```
>>> kk.set_intent(2,'xx.')
```
or
```
>>> kk.set_intent(2,{'aquatic','mammal'})
```
(there are more possibilities)

However, this only works if the variable `kk` has been assigned a formal context for which object and attribute names have been defined. Only the middle command does not require a string name. The string `'xx.'` represents the incidences of the object `2`, as in the printed context.

Incidentally, there is a shortcut for all of this: the command `cxt_typein()`. It asks all the necessary questions; the answers are entered without apostrophes, even if they are strings. The following lines show the input of the "Animals" context. The green text comes from the program. User input is in black.

```
>>> anim = cxt_typein()
Number of objects: 3
Number of attributes: 3
Context name: Animals
Name of attribute 0 (default = 0): aquatic
Name of attribute 1 (default = 1): mammal
Name of attribute 2 (default = 2): predator
Name of object 0 (default = 0): lynx
Intent of this object: .xx
Name of object 1 (default = 1): shark
Intent of this object: x.x
Name of object 2 (default = 2): manatee
Intent of this object: xx.
>>>
```

31 Some Python for formal contexts

- Defining, loading, or typing a formal context.

```
>>> kk = Context(gnum,mnum)
>>> kk = Context.read(filename)
>>> kk = cxt_typein()
```



- Modifying a formal context "on the spot" (no return value).

```
>>> kk.make_incident(g,m)
>>> kk.make_nonincident(g,m)
>>> kk.set_intent(g,intent)
>>> kk.gname[object number] = string
>>> kk.mname[attribute number] = string
>>> kk.name = string
```

- Retrieving basic information

```
>>> kk.is_incident(g,m)
>>> kk.number_of_concepts()
```

- Methods returning a modified context.

```
>>> kk2 = kk1.clarify()
>>> kk2 = kk1.reduce()
>>> kk2 = kk1.add_object(intent,[object name])
```

- Making a new context from a given one.

```
>>> kk2 = kk1.copy()
>>> kk2 = kk1.dual()
>>> kk2 = kk1.complement()
>>> kk2 = kk1.subcontext(gset,mset)
```

- Arrow relations.

```
>>> da = kk.down_arrows()
>>> ua = kk.up_arrows()
```

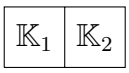
Figure 6: Simple methods for formal contexts.

32 Creating a formal context from two given ones

For details, see the `fca_math` package documentation.

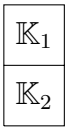
- Apposition

```
>>> kkapp = kk1 | kk2
```



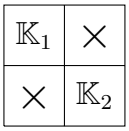
- Subposition

```
>>> kksub = kk1 / kk2
```



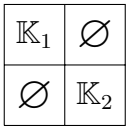
- Context sum

```
>>> kksum = kk1 + kk2
```



- Disjoint union

```
>>> kkunion = kk1.disjoint_union(kk2)
```



- Context product

```
>>> kkprod = kk1 * kk2
```

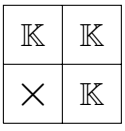
- Semiproduct

```
>>> kksemi = kk1.semiproduct(kk2)
```



Figure 7: Formal context composition methods.

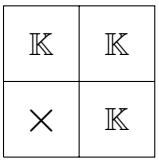
Exact definitions can be found in the book mentioned in Section 13. These context constructions can be iterated, giving you the option of setting up new and interesting constructions. Here's an example. Starting from a formal context $\mathbb{K}$, a context $\mathbb{K}_{ord}$ is obtained as indicated.



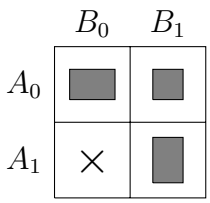

```
>>> full = Context(kk.gnum,kk.mnum).complement()
>>> kkord = (kk | kk) / (full | kk)
```

Its conceptual interpretation is discussed in the next section.

33 The order relation of a complete lattice



What are the formal concepts of the context $\mathbb{K}_{ord}$, which was introduced in the previous section and is shown graphically on the right? In order to answer this question properly, we first need to be a little more precise. In the small sketch, $\mathbb{K}$ appears several times, but of course different rows must not refer to the same objects and different columns must not refer to the same attributes. This can be made precise by adding indices to objects and attributes, but we shall ignore this technicality here.



Consider a formal concept $(A_0 \dot\cup A_1, B_0 \dot\cup B_1)$ of $\mathbb{K}_{ord}$, i.e., a maximal full subrectangle of the incidence relation. Then $A_0 \times B_0$ is a maximal full rectangle in the upper left quadrant, and $A_1 \times B_1$ a maximal full rectangle in the south east. So both correspond to formal concepts of $\mathbb{K}$. But note that $A_0 \times B_1$ must be "full of crosses" (i.e., a subset of the incidence), and that requires that $B_1 \subseteq B_0$ and $A_0 \subseteq A_1$.

For $(A_0 \dot\cup A_1, B_0 \dot\cup B_1)$ to be a formal concept of $\mathbb{K}_{ord}$, it thus is necessary and sufficient, that (A_0, B_0) and (A_1, B_1) are formal concepts of $\mathbb{K}$ with $(A_0, B_0) \leq (A_1, B_1)$.

In fact, for every complete lattice $\boldsymbol{L} := (L, \leq)$, the order relation, i.e., the set of all pairs (v, w) with $v \leq w$, itself forms a complete lattice. The order is $(v_1, w_1) \leq (v_2, w_2) \iff v_1 \leq v_2$ and $w_1 \leq w_2$. The context construction discussed above shows how a formal context for the order relation can be obtained when $\boldsymbol{L}$ is the concept lattice of $\mathbb{K}$.

This construction can be formulated even more elegantly by using the context product. The mathematical definition of the product can be easily visualized using the imagery of crosstables. To form the product of $\mathbb{K}_1$ and $\mathbb{K}_2$, first enlarge each "cell" of $\mathbb{K}_2$ to the size of $\mathbb{K}_1$. Then write a copy of $\mathbb{K}_1$ into each empty cell of $\mathbb{K}_2$ and replace each full cell with a full context of the corresponding size, i.e., fill it with crosses.

ℂ	0	1
0		
1	×	

This visualization provides exactly the result shown at the top right of this page, when forming the context product of $\mathbb{K}$ with the context $\mathbb{C}$. Replace each of the three empty cells of $\mathbb{C}$ by a copy of $\mathbb{K}$, and replace the non-empty cell by a "full" context of the size of $\mathbb{K}$, and the result is $\mathbb{K}_{ord}$.

Using the `fca_math` package, this can be achieved as follows:

```
>>> cc = Context(2,2).make_incident(1,0)
>>> kkord = kk * cc
```

34 Quick results with simple programs

34.1 Counting monotone Boolean functions

A famous counting problem, which goes back to the mathematician Richard Dedekind, asks for the number of monotonic Boolean functions in n variables. These "Dedekind numbers" are known up to $n = 10$. The tenth is a forty-two digit decimal number. More can be found in Section 35

It is also known that these functions are in bijective correspondence with the concepts of the *free completely distributive lattice* with n generators, and that this lattice is isomorphic to the concept lattice of the nth power of the 2×2 context with only single-element incidence. Here is our computation for the case $n = 5$:

```
>>> cc = Context(2,2)
>>> cc.make_incident(1,0)
>>> (cc * cc * cc * cc * cc).number_of_concepts()
7581
```

There are 7581 monotone Boolean functions in five variables.

Too technical? Take a look at Section 36!

34.2 Size of the order relation

The powerset $\mathfrak{P}(X)$ of a set X, i.e., the set of all subsets of X, ordered by set inclusion $\subseteq$, is a complete lattice and thus isomorphic to a concept lattice. In fact, the corresponding standard context for the case $|X| = n$ is precisely the *contranominal scale* with n objects and attributes, which is predefined as `Scale.contranominal(n)`. Therefore, with

```
>>> cn4 = Scale.contranominal(4)
```

we obtain a formal context `cn4` with $2^4 = 16$ concepts, as can be quickly verified:

```
>>> cn4.number_of_concepts()
16
```

Now let's ask a more difficult question: How many *pairs* of subsets (S, T) for which $S \subseteq T$ holds are there of a four-element set? In other words, what is the size of the order relation of the powerset (of a four-element set)? According to Section 33, the size of the order relation is determined as follows (`cn4` is the contranominal scale from above):

```
>>> cc = Context(2,2); cc.make_incident(1,0)
>>> cn4ord = cn4 * cc
>>> cn4ord.number_of_concepts()
81
```

So the result is that there are $81 = 3^4$ such pairs of sets. Can you prove a general rule? And can you explain why `cn4 * cc * cc` has 6^4 elements?

35 Side note: Monotone Boolean functions

The program examples presented in Section 34 were not easy to understand because some mathematical magic was used. If that didn't bother you, you can skip this section and the next one as well. Here, we review some basic knowledge about Boolean functions. This is the basis for understanding the examples given in Section 36

In elementary Propositional Logic, the symbols **T** and **F** denote the *truth values* "true" and "false". They are naturally ordered by $\mathbf{F} < \mathbf{T}$.

An n-ary Boolean function (also called a Boolean function *in n variables*) assigns such a truth value to every n-tuple of **T**s and **F**s. Here is an example for $n = 2$, known as the **Peirce function** or as **NOR**:

$$(\mathbf{F},\mathbf{F}) \mapsto \mathbf{T}, \quad (\mathbf{F},\mathbf{T}) \mapsto \mathbf{F}, \quad (\mathbf{T},\mathbf{F}) \mapsto \mathbf{F}, \quad (\mathbf{T},\mathbf{T}) \mapsto \mathbf{F}.$$

Quite obviously, the number of such n-ary Boolean functions is 2^{2^n}. They can be ordered:

$$f \le g :\iff f(v_1,\dots,v_n) \le g(v_1,\dots,v_n) \text{ for all } v_1,\dots,v_n \in \{\mathbf{F},\mathbf{T}\}.$$

And yes, with this order they form a complete lattice that is isomorphic to the powerset lattice of a 2^n-element set.

An n-ary function $f(v_1,\dots,v_n)$ is **monotone** if $v_1 \le w_1$, $v_2 \le w_2$, …, $v_n \le w_n$ together imply that $f(v_1,\dots,v_n) \le f(w_1,\dots,w_n)$.

Note that every n-ary Boolean function can be decomposed in two $(n-1)$-ary ones by

$$f(v_1,\dots,v_n) = \begin{cases} g(v_1,\dots,v_{n-1}) & \text{if } v_n = \mathbf{F} \\ h(v_1,\dots,v_{n-1}) & \text{if } v_n = \mathbf{T} \end{cases}.$$

Conversely, this can also be used as a construction method to obtain an n-ary function from two $(n-1)$-ary functions. The following is easy to prove:

Proposition 4 *The following are equivalent:*

1. *f is monotone,*
2. *g and h are both monotone and $g \le h$.*

This establishes a 1-1 correspondence between the n-ary monotonic Boolean functions and the pairs (g,h) of $(n-1)$-ary Boolean functions with $g \le h$. Since g and h can be reconstructed from f, the relationship is bijective. It is even an order-isomorphism: If f_1 is obtained from g_1 and h_1, and f_2 is obtained from g_2 and h_2, then $f_1 \le f_2$ iff both $g_1 \le g_2$ and $h_1 \le h_2$.

tl;dr: The ordered set of all n-ary monotone Boolean functions is order-isomorphic to the set of all pairs (g,h), $g \le h$, of $(n-1)$-ary monotone Boolean functions with componentwise order:

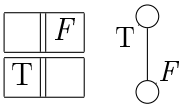
$$(g_1,h_1) \le (g_2,h_2) \iff g_1 \le g_2 \text{ and } h_1 \le h_2.$$

36 Understanding 34.1

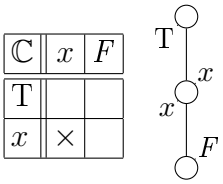
Now we describe the ordered set of all n-ary monotone Boolean functions for $n = 0, 1, 2, 3$.

Q: Wait a minute, $n = 0$? What is a zero-ary Boolean function?

A: Zero-ary functions ("in zero variables") do not depend on any input and therefore must be constant. In the present case, they can only be the constant function $\mathbf{F}$, or the constant function $\mathbf{T}$.

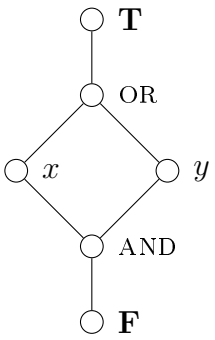


The two constant Boolean functions $\mathbf{F}$ and $\mathbf{T}$ form a two-element lattice, since $\mathbf{F} < \mathbf{T}$.



The order relation of this two element lattice has three elements: $(\mathbf{F}, \mathbf{F})$, $(\mathbf{F}, \mathbf{T})$, and $(\mathbf{T}, \mathbf{T})$. With the componentwise order, these form a three-element chain. The standard context is just the context $\mathbb{C}$, which was defined in Section 33. Two of the three monotone Boolean functions in one variable are constant, only one is not. We call it x.

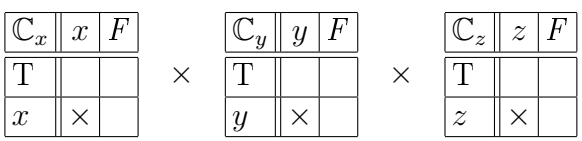
The order relation of the three-chain $\{\mathbf{F} < x < \mathbf{T}\}$ has six elements: $\{(\mathbf{F}, \mathbf{F}), (\mathbf{F}, x), (\mathbf{F}, \mathbf{T}), (x, x), (x, \mathbf{T}), (\mathbf{T}, \mathbf{T})\}$. These correspond to the six monotone Boolean functions $f(x, y)$ in two variables: $(\mathbf{F}, \mathbf{F})$ stands for the constant function $\mathbf{F}$, $(\mathbf{F}, x)$ for the function $f(x, y)$ with $f(x, \mathbf{F}) = \mathbf{F}$ and $f(x, \mathbf{T}) = x$ for $x \in \{\mathbf{F}, \mathbf{T}\}$, i.e., the AND function, $(\mathbf{F}, \mathbf{T})$ denotes the function $f(x, y)$ that returns $\mathbf{F}$ whenever y is $\mathbf{F}$ and $\mathbf{T}$ elsewhere, thus it simply denotes $f(x, y) = y$. Et cetera. The standard context is the context product of $\mathbb{C}$ with itself. The lattice of monotone Boolean functions in two variables is shown on the right. It is labeled by the function names, not by context objects and attributes.



Q: Where are the other functions, like XOR, NAND, etc.?

A: They are not monotone!

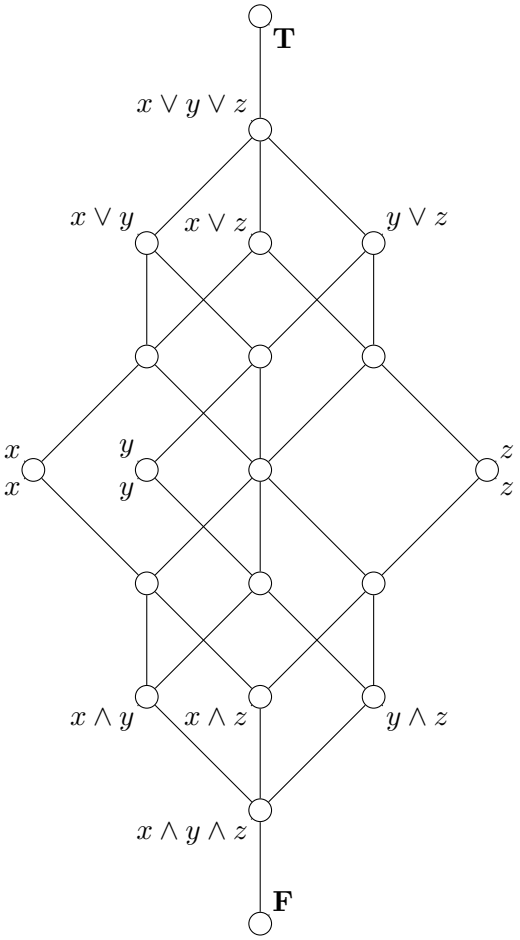
Finally, we show the lattice of monotone Boolean functions in three variables, which we call x, y and z. We already know that it can be obtained as the order relation of the lattice of binary monotone Boolean functions, and therefore also as the context product $\mathbb{C} \times \mathbb{C} \times \mathbb{C}$. For better readability, we rename and use the following product:



The result is shown on the next page.

37 A diagram of $FCD(3)$

$FCD(3)$	$x \vee y \vee z$	$x \vee y$	$x \vee z$	x	$y \vee z$	y	z	**F**
T								
z	×		×		×		×	
y	×	×			×	×		
$y \wedge z$	×	×	×		×	×	×	
x	×	×	×	×				
$x \wedge z$	×	×	×	×	×		×	
$x \wedge y$	×	×	×	×	×	×		
$x \wedge y \wedge z$	×	×	×	×	×	×	×	



The diagram shows the concept lattice of the 20 monotone Boolean functions in the variables x, y, z. When labeling, the tuples of objects were written as conjunctions and those of attributes as disjunctions. The concept extents and intents can thus be understood as sets of conjunctions or disjunctions. When interpreting the extents as *disjunctions* of conjunctions and the intents as *conjunctions* of disjunctions, every formal concept describes the corresponding Boolean function by a DNF and a CNF.[7]

Example: There are three unlabeled circles on the vertical middle axis of the diagram. The lowest of these represents the formal concept

$$(\{x \wedge y \wedge z, x \wedge y, y \wedge z\}, \{y, x \vee y, x \vee z, y \vee z, x \vee y \vee z\}),$$

or, when written as DNF and CNF,

$$((x \wedge y \wedge z) \vee (x \wedge y) \vee (y \wedge z), y \wedge (x \vee y) \wedge (x \vee z) \wedge (y \vee z) \wedge (x \vee y \vee z)).$$

These expressions can be simplified, the monotone Boolean function is

$$f(x, y, z) = y \wedge (x \vee z).$$

[7] Disjunctive Normal Form, Conjunctive Normal Form, here without negations

38 Nerd stuff: Beyond 34.2

In 34.2, we determined the size of the order relation for the power set of a four-element set. To do this, we simply multiplied the contra-nominal scale by the 2×2 context `cc` and then determined the number of formal concepts. The result was 81, but some questions remained unanswered and will be answered here.

Q: Is it generally true that the order relation of the power set of an n-element set has 3^n elements?

A: Yes. To count the number of pairs (A, B) with $A \subseteq B \subseteq \{1, \ldots, n\}$, associate to each such pair a mapping $f_{(A,B)} : \{1, \ldots, n\} \to \{0, 1, 2\}$ by

$$f(x) = \begin{cases} 1 & \text{if } x \in A \\ 2 & \text{if } x \in (B \setminus A) \\ 0 & \text{else.} \end{cases}$$

This encoding is bijective, and there are obviously n^3 such mappings. The order relation as a lattice is isomorphic to the nth power of a three-element chain.

Q: If you iterate and count the order relation of the order relation, you get 6^4 in the example. Is there a general rule behind this, and is that lattice always the nth power of a 6-element chain?

A: Yes and no. The number of elements is 6^n, but it is *not* a power of the 6-element chain.

This is best understood algebraically: The smallest contranominal scale `Scale.contranominal(1)` is the 1×1 context with empty incidence, i.e., without a cross. Let's call it `o1` here. The n-th contranominal scale is isomorphic to the n-fold sum of `o1`, i.e.

$$\texttt{Scale.contranominal(n)} \cong \texttt{o1} + \texttt{o1} + \ldots + \texttt{o1}.$$

Recall from 34.2 that the formal context of the order relation is obtained by multiplying with `cc`, the 2×2-context with one cross. The order of the order of the nth powerset is therefore the concept lattice of

$$(\texttt{o1} + \texttt{o1} + \ldots + \texttt{o1}) * \texttt{cc} * \texttt{cc}.$$

It can be shown that the *distributive law*

$$(\mathbb{K}_1 + \mathbb{K}_2) * \mathbb{K}_3 = \mathbb{K}_1 * \mathbb{K}_3 + \mathbb{K}_2 * \mathbb{K}_3$$

holds an that `o1` is a neutral element for context multiplication. This yields

$$\texttt{cn4ordord} = \texttt{cc} * \texttt{cc} + \texttt{cc} * \texttt{cc} + \texttt{cc} * \texttt{cc} + \texttt{cc} * \texttt{cc}.$$

The concept lattice is (isomorphic to) the fourth power of the six-element lattice shown on Page 33.

Contents

1 First questions and answers 2

2 What the reader should already know 2

3 Getting started 3

4 Understanding the Anorexia data 4

5 Derivation 4

6 Formal concepts 5

7 Style guide 6

8 Questions and answers 6

9 Concept lattice 7

10 A smaller example: stirrers 7

11 Concept lattice diagrams 8

12 A diagram for the Anorexia data 10

13 Two angry questions answered by an advertisement 11

14 Python programs for Formal Concept Analysis 12

15 First computations 13

16 Drawing a concept lattice diagram by hand 14

17 Where the labels go 15

18 Style guide (diagrams) 16

19 Exercises 16

20 Computing with extents and intents 17

21 Complete Lattices and the Basic Theorem 18

22 Style guide (Basic Theorem) 19

23 Questions and answers (Basic Theorem) 20

24 Clarifying a formal context 21

25 Reducing a formal context 22

26 Questions and answers (clarifying and reducing) 23

27 Finger exercises: Fun with tiny lattices 24

28 Standard contexts available for `fca_math` 25

29 Editing a formal context 26

30 Make context manipulation more convenient 27

31 Some `Python` for formal contexts 28

32 Creating a formal context from two given ones 29

33 The order relation of a complete lattice 30

34 Quick results with simple programs 31
34.1 Counting monotone Boolean functions 31
34.2 Size of the order relation 31

35 Side note: Monotone Boolean functions 32

36 Understanding 34.1 33

37 A diagram of $FCD(3)$ 34

38 Nerd stuff: Beyond 34.2 35